neueda

Exception Handling in Java

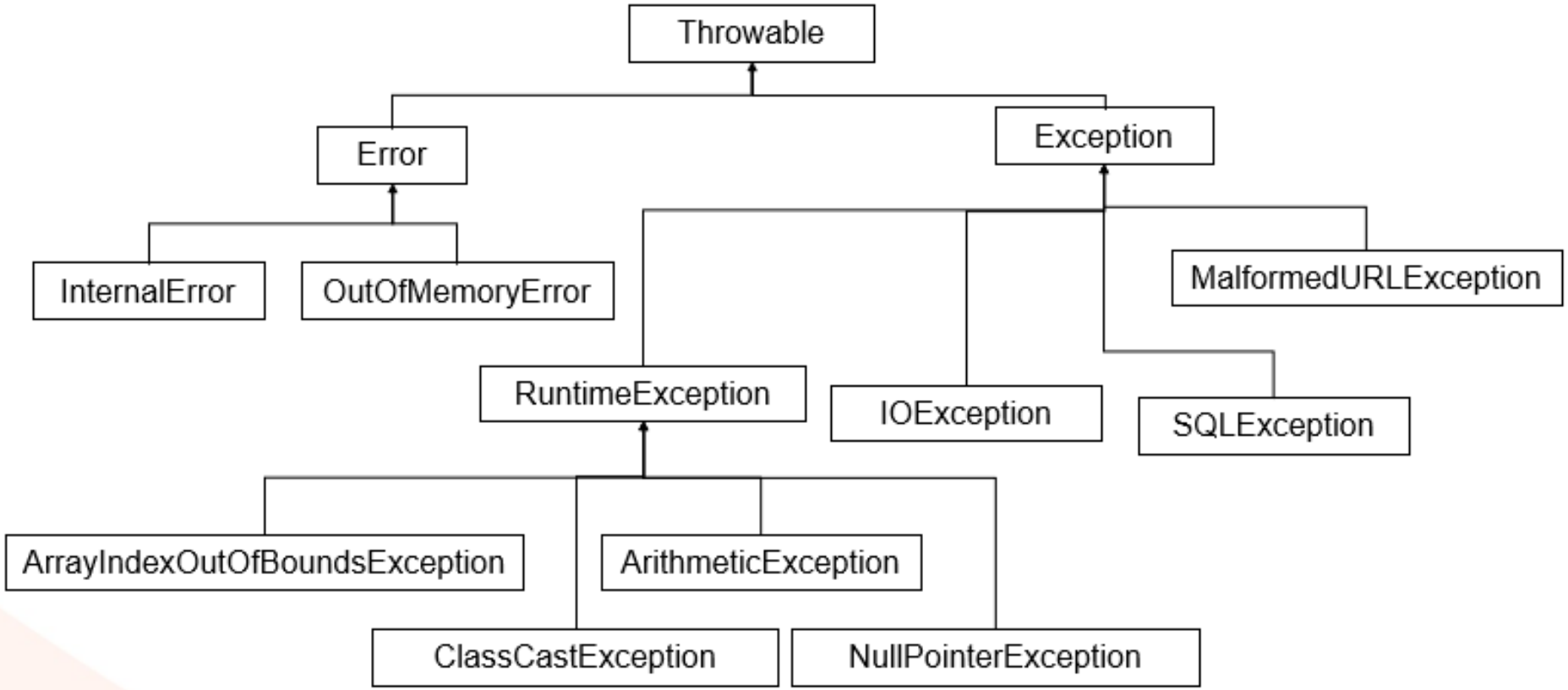
Objectives

- Errors and Exceptions
- Runtime Exceptions
- Checked Exceptions
- Try blocks
- Catch blocks
- Finally blocks
- Throwing Exceptions
- Defining custom exceptions
- Logging

Errors and Exceptions

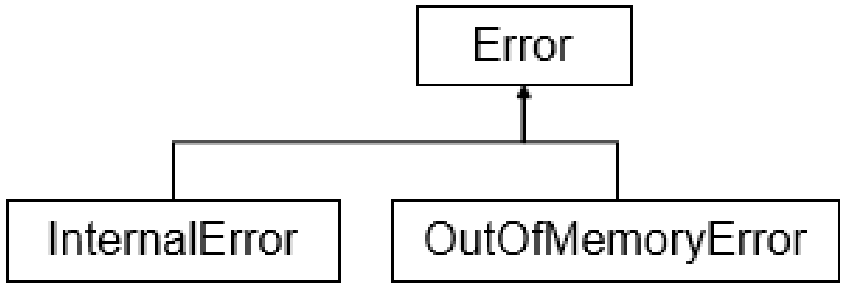
- Some languages use error codes
- In Java, Errors and Exceptions are objects
- Errors and Exceptions are *thrown* and *caught* by blocks of code
- There is a class hierarchy of errors and exceptions in Java
- All extend Throwable

Throwable Hierarchy



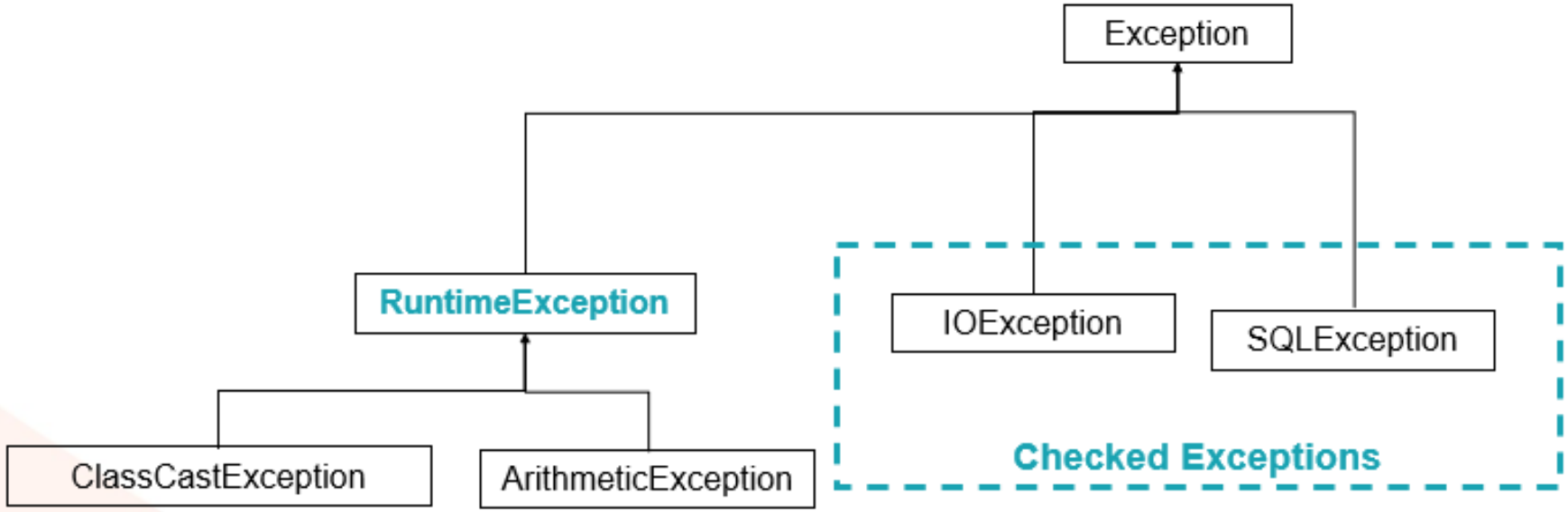
Errors

- There is little you need to do about Errors
- If the VM finishes inappropriately, you cannot recover from it
- The error is thrown, but nothing else needs to be done by you



Exceptions

- There are two categories of Exception
  1. Unchecked / Runtime Exceptions
  2. Checked Exceptions



Runtime Exceptions

- The Java built in runtime exceptions are those that are avoided by writing good defensive code

Exception example	Exception cause	Exception detail
NullPointerException	Use a non-existent object	String name = null; name.toUpperCase();
ArithmeticException	Divide an integral number by zero	int infinity = 10 / 0;
ArrayIndexOutOfBounds Exception	Access a non-existent array index	int[] lotto = new int[6]; lotto[10] = 5;

Checked Exceptions

- The built in checked exceptions are those that the developer has no direct control over, and can be recovered from

Exception example	Exception cause
IOException	Read from a nonexistent file
SQLException	Access a non-existent table
MalformedURLException	Create a URL object with a malformed protocol

Handling Checked Exceptions

- You HAVE TO deal with checked exceptions in your code one way or another
- Consider the following database connection code

```
Connection conn = DriverManager.getConnection(“database URL”);
```

- This connection may fail
- You must do something about this
- One option is put this line of code into what is referred to as a *try / catch block*

Try Blocks

- Any line of code that may *throw* an exception can be placed inside a *try* block

```
try {
    Connection conn = DriverManager.getConnection(“database URL”);
}
```

- You are saying in effect ‘*try this*’
- If it fails, the path of execution falls to an appropriate *catch* block

Catch Blocks

- This block will *catch* the exception and deal with it accordingly

```
try {
      Connection conn = DriverManager.getConnection("database URL");
}

catch ( SQLException e) {
      System.out.println ("A connection could not be established" + e);
}
```

Exception Methods

- Exception classes have some useful methods

Exception example	Exception action
toString()	Returns a String representation of the exception.
getMessage()	Returns the detail message of the exception.
printStackTrace()	Prints the stack trace leading up to the exception.

```
catch ( SQLException e) {
      System.out.println (“Exception: “ + e);
}
```

Clean Up

- There may be certain statements that need to be executed whether an exception occurred or not
  - Eg. whilst connected to a database, you would need to close the connection whether an error occurred or not
- These statements are referred to as clean up
- They can be placed in a ***finally*** block

Finally Blocks

- Finally will *always* be executed

```
Connection conn = null;
try {
      conn = DriverManager.getConnection(“database URL”);
}

catch ( SQLException e) {
      System.out.println (“A connection could not be established”);
      return; // will execute the finally block before it returns
}

finally {
      // this will always be executed
      conn.close(); // slightly simplified as needs to be in nested try/catch
}
```

Finally – an Alternative ‘try with resources’

- Java 7 introduced a new syntax referred to as **try-with-resources**
  - This new construct allows resources such as streams (and database connections) to be closed automatically at the end of a try block

```
try (
   FileReader reader = new FileReader("input.txt");
   BufferedReader buffReader = new BufferedReader(reader);
   FileWriter writer = new FileWriter("output.txt");
   BufferedWriter buffWriter = new BufferedWriter(writer);
) {
    writer.write("here is the output\n");
    while(true) {
       String line = buffReader.readLine();
        if (line == null)
            break;
       buffWriter.write(line + "\n");
} // everything in the try() above is automatically closed at this point
```

How Autoclosing Works

- All classes that can be used in this way implement an interface called **AutoClosable** which has a method called close()
  - **close()** is called automatically in the same way a finally block is called

```
public class MyCloseable implements AutoCloseable {
  @Override
  public void close() throws Exception {
    System.out.println("Closing my thing!");
  }
}
```

Passing Exceptions (1)

- In this method, the body determines what to do with the exception if thrown

```
public void getDatabaseConnection() {
      try {
          Connection conn = DriverManager.getConnection("database URL");
      }

      catch ( SQLException e) {
          System.out.println ("A connection could not be established" + e);
      }
}
```

Passing Exceptions (2)

- The catch block content sometimes depends on who the *caller* of the method is
- In these cases we can '*pass the buck*'

```
public Connection getDatabaseConnection() throws SQLException {
        Connection conn = DriverManager.getConnection("database URL");
        return conn;
}
```

- The caller of the method now has to put the call inside a try / catch block

Calling Methods that throw Exceptions

- The method call is inside a try / catch

```
try {
    Connection conn1 = objectRef.getDatabaseConnection();
}
catch ( SQLException e) {
    System.out.println ("A connection could not be established" + e);
}
```

- Methods can throw more than one type of exception; they are separated by commas

```
public Connection  getDatabaseConnection() throws SQLException, IOException
```

Inheritance and Exceptions

- Subclass methods which have overridden superclass methods cannot throw more exceptions than the superclass method

Superclass

```
public void getDatabaseConnection() throws SQLException
```

Subclass

✖

```
public void getDatabaseConnection() throws SQLException, IOException
```

Defining Your Own Exceptions

- To use your own exception classes
  1. Subclass the appropriate Exception class (usually Exception)
  2. Explicitly throw instances of your new exception class from your code

Defining Exception Classes

- Subclass Exception

```
public class DuplicateCustomerException extends Exception {
    public DuplicateCustomerException (String s)  {// takes in the message
        super (s);
    }
    public DuplicateCustomerException (Exception rootCause, String s) {
        super(rootCause, s);
}
    public String toString() {
        return “A duplicate customer was found with id ….”
            +  “Root cause was” + getCause();
    }
}
```

Throwing An Exception

- Use the *throw* keyword

```
public void someMethod throws DuplicateCustomerException {
   ….
   catch (SQLException e)  {
      throw new DuplicateCustomerException(e, “Customer already there”);
   }
}
```

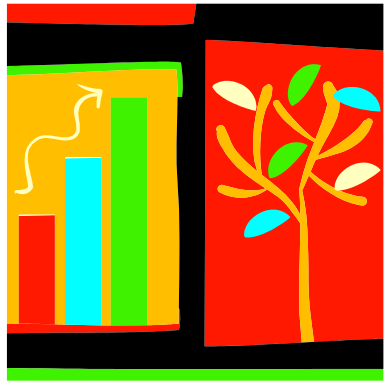
Logging in Java

- On the Java platform there are three popular logging libraries
  - Log4J
  - Commons Logging
  - Java Built in Logging (since Java 1.4)

- Since the Java built in logging came about relatively late, many APIs use Log4J or Commons Logging

Java Built in Logging

- Java 1.4 introduced the new package java.util.logging
- Using the Logger class you can implement logging
- Logs can be in two formats
  - XML
  - Plain text
- Logs can be sent to
  - Files
  - Streams
  - Console
  - Memory
  - Sockets



Logging Example

- Logging example code

```
Logger logger = Logger.getLogger("LoggingExample");
logger.setLevel(Level.INFO);
try {
  FileHandler handler = new FileHandler("logfile.xml");
  handler.setFormatter(new XMLFormatter());
  logger.addHandler(handler);
}
catch (IOException e)  {
  System.out.println("IOException " + e);
}
logger.severe ("Here is a SEVERE message");
logger.warning("Here is a WARNING message");
}
```

The XMLFormatter Output

- The output of the XMLFormatter is shown

```
<log>
<record>
  <date>2004-10-20T21:59:48</date>
  <millis>1098305988359</millis>
  <sequence>0</sequence>
  <logger>LoggingExample</logger>
  <level>SEVERE</level>
  <class>LoggingExample</class>
  <method>main</method>
  <thread>10</thread>
  <message>Here is a SEVERE
     message</message>
</record>
..
</log>
```

Summary

- Errors and Exceptions
- Runtime Exceptions
- Checked Exceptions
- Try blocks
- Catch blocks
- Finally blocks
- Throwing Exceptions
- Defining custom exceptions
- Logging